Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : CSA0927 – Programming in Java

Assignment Title:	Government Public Grievance Management System using Java
Course Outcome(s) – CO:	**CO1:** Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4) **CO2:** Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4) **CO3:** Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques (BL3,BL4)
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 16 – Peace, Justice and Strong Institutions SDG 9 – Industry, Innovation and Infrastructure SDG 10 – Reduced Inequalities SDG 11 – Sustainable Cities and Communities

Problem Statement : A district government office receives numerous public complaints related to roads, water supply, sanitation, streetlights, public transport, and civic services. The existing manual process makes it difficult to register complaints, assign them to the appropriate departments, track their status, and monitor pending cases. The government department plans to develop a Java-based Public Grievance Management System to automate the complaint-handling process. The system should maintain citizen details, complaint records, departments, responsible officers, priority levels, and complaint status.Different users such as citizens, officers, supervisors, and administrators should have different roles and responsibilities. The application should apply classes, encapsulation, inheritance, method overriding, and polymorphism to represent these entities.Citizen records, complaints, department details, and complaint history should be managed using the Java Collection Framework, including ArrayList, HashSet, and HashMap, along with generics and iterators.Since multiple citizens may submit complaints simultaneously, the system should implement multithreading, synchronization, and inter-thread communication to process requests safely and avoid conflicts. Appropriate exception handling should be provided for duplicate complaints, invalid inputs, unavailable departments, and incorrect status updates.The final application should provide a menu-driven interface for citizen registration, complaint submission, officer assignment, status updates, complaint tracking, and generation of department-wise and pending-complaint reports.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
1. Requirement Analysis, Java Fundamentals & Program Logic	**15**	Clearly identifies citizens, grievances, departments, officers, priorities, and workflows; uses appropriate data types, operators, conditional statements, loops, and decision-making structures correctly and efficiently.	Most requirements and workflows are identified; Java fundamentals and control structures are used correctly with minor errors.	Basic requirements are identified; fundamental constructs are used with some logical errors or inefficiencies.	Requirements are poorly understood; frequent errors in Java fundamentals and program logic affect correctness.

2. Class Design, Encapsulation, Inheritance & Polymorphism	**20**	Well-structured classes for citizens, officers, departments, grievances, and management; strong encapsulation and data hiding; appropriate inheritance, method overriding, and runtime polymorphism are implemented effectively.	Classes are reasonably designed with appropriate encapsulation and inheritance; polymorphism is implemented with minor limitations.	Basic classes and inheritance are present, but encapsulation or polymorphic behavior is incomplete.	Poor class organization; weak encapsulation; inheritance or polymorphism is absent or incorrectly implemented.
3. Java Collection Framework, Generics & Data Management	**20**	Effectively uses `List`, `ArrayList`, `Set`, `HashSet`, `Map`, and `HashMap` with generics and iterators to store, search, update, and process citizen, grievance, department, and complaint-history records.	Appropriate collections and generics are used with minor implementation or retrieval issues.	Collections are used for basic storage and retrieval, but some data structures or iterator operations are incomplete.	Collections, generics, or iterators are largely missing, incorrectly implemented, or inefficiently used.
4. Multithreading, Synchronization, Exception Handling & Menu-Driven Functionality	**25**	Multiple threads effectively process concurrent grievance submissions, officer assignments, and status updates; synchronization and inter-thread communication prevent conflicts; exception handling manages duplicate complaints, invalid inputs, unavailable departments, and incorrect status updates; all menu	Multithreading, synchronization, exception handling, and menu operations are mostly implemented correctly with minor issues.	Basic multithreading and exception handling are present, but synchronization, communication mechanisms, or some menu functions are incomplete.	Multithreading or synchronization is absent/non-functional; exception handling is inadequate and major menu operations are missing or unreliable.

		operations function reliably.			
5. System Integration, Reports, Documentation & Overall Quality	**10**	All modules are fully integrated; complaint tracking, departmental allocation, pending-grievance analysis, and performance reports are generated correctly; code is organized, readable, documented, and professionally presented.	Modules are well integrated and reports are mostly correct; code organization and documentation are satisfactory.	Basic integration and reporting are achieved, but some reports, documentation, or code quality need improvement.	Poor integration; reports are missing or incorrect; code is difficult to understand and documentation is inadequate.
6. Reflection: Design Decisions, SDG Relevance & Learning Outcomes	**10**	Provides thoughtful justification of design decisions; clearly connects the system with **SDG 16 – Peace, Justice and Strong Institutions**, along with relevant SDGs such as SDG 9, 10, and 11; provides specific reflection on challenges, problem-solving, and learning outcomes.	Provides reasonable justification of design choices and SDG relevance; discusses challenges and learning with adequate detail.	Reflection is present but generic; limited justification of design decisions, SDG relevance, or learning outcomes.	Reflection is missing or lacks meaningful discussion of design decisions, SDG relevance, challenges, or learning outcomes.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

GOVERNMENT GRIEVANCE MANAGEMENT SYSTEM

1. Problem Statement

Citizens often face difficulties in submitting and tracking complaints related to public services such as roads, water supply, sanitation, streetlights, public transport, and other civic services. Manual grievance handling can result in delays, incorrect department assignment, duplicate complaints, and difficulty in tracking complaint status.

The Government Grievance Management System is developed as a Java-based application to provide a centralized platform for registering citizens, submitting grievances, automatically assigning complaints to appropriate departments, assigning officers, tracking complaint status, and generating reports. The system also demonstrates Java concepts such as object-oriented programming, collections, exception handling, multithreading, synchronization, and wait()/notifyAll().

2. Objective

- To provide an organized system for registering citizens.
- To allow citizens to submit public grievances.
- To categorize complaints based on service type.
- To assign complaints automatically to the appropriate government department.
- To assign responsible officers to complaints.
- To track the current status and history of complaints.
- To identify and prevent duplicate complaints.
- To generate pending, department-wise, status-wise, and priority-wise reports.
- To demonstrate Java Collection Framework concepts.
- To demonstrate multithreading and synchronization.
- To provide a simple and user-friendly graphical interface.

3. Requirements and Environment Used

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Minimum 10 GB available storage
- Keyboard and mouse
- Display monitor

Software Requirements

Requirement	Details
Programming Language	Java
JDK	JDK 21
IDE	Eclipse IDE
GUI Technology	Java Swing
Database	Not required
Operating System	Windows / Linux / macOS
Collections	ArrayList, HashSet, HashMap, Queue
Threading	Java Threads
Version Control	Github

The application is implemented as a standalone Java Swing application, so an external database is not required for the current implementation.

4. Design / Proposed Solution

The proposed system consists of several modules that work together to manage public grievances.

Module	Description
Citizen Management	Allows citizens to be registered with Citizen ID, Name, Phone number, Email, and Address. The

	system validates citizen information and prevents duplicate citizen IDs.
Complaint Management	Allows citizens to submit complaints by providing Complaint ID, Citizen ID, Complaint category, Priority, and Complaint description.
Department Management	Maintains government departments such as Roads, Water Supply, Sanitation, Streetlight, Public Transport, and Civic Services.
Officer Management	Associates government officers with departments and allows officers to be assigned to complaints.
Automatic Department Assignment	Assigns a complaint to the appropriate department according to its category.
Complaint Tracking	Maintains complaint status and history with remarks and date/time of updates.
Reports and Analytics	Generates pending complaint, department-wise, status-wise, and priority-wise reports.
Multithreading	Demonstrates concurrent complaint processing using multiple threads, synchronization, wait(), and notifyAll().

Category-to-department mapping:

Complaint Category	**Department**
ROADS	Roads Department
WATER_SUPPLY	Water Supply Department
SANITATION	Sanitation Department
STREETLIGHT	Streetlight Department
PUBLIC_TRANSPORT	Public Transport Department
CIVIC_SERVICES	Civic Services Department

DESIGN / PROPOSED SOLUTION

Proposed Solution

The proposed Government Grievance Management System is a Java Swing desktop application organized around a simple end-to-end grievance workflow. A user enters citizen and complaint details, the system validates the information, generates a complaint identifier, assigns a department and stores the record. Staff can then search the complaint and update its status.

Proposed System Architecture

The system architecture is intentionally lightweight and suitable for a single Java source-file implementation:

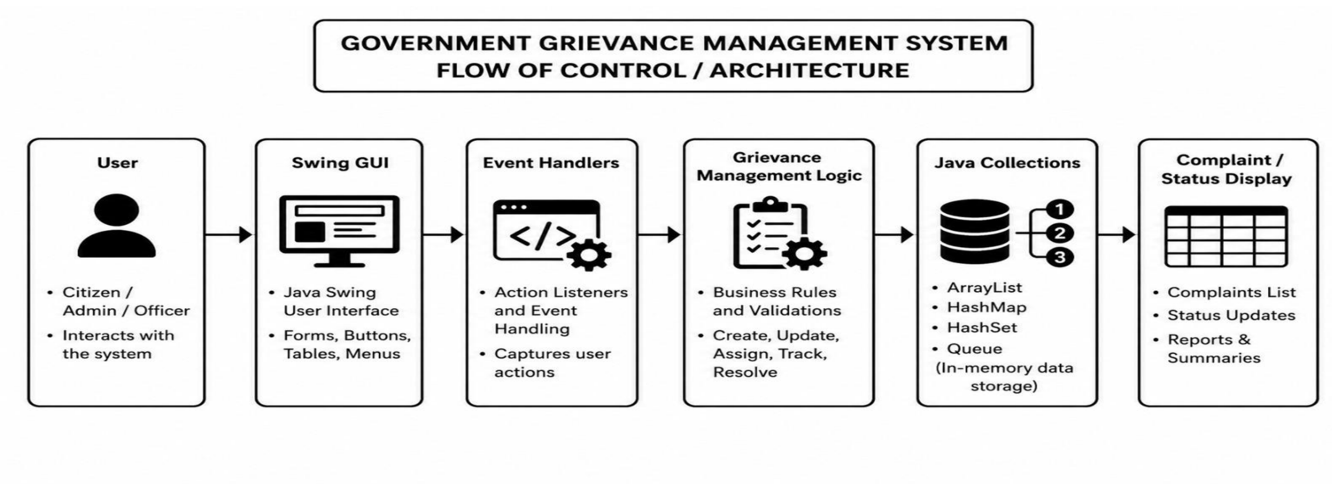


FIG 01 : Government Grievance Management System Architecture

The architecture can be represented as: User → Swing GUI → Event Handlers → Grievance Management Logic → Java Collections → Complaint/Status Display.

5. ALGORITHM / PSEUDOCODE / FLOWCHART

Algorithm

1. Start the application.
2. Initialize grievance collections and predefined departments/categories.
3. Display the main graphical interface.
4. Accept citizen details and grievance information.
5. Check whether mandatory fields are completed.

6. If validation fails, display an error message and return to the form.
7. Generate a unique complaint ID.
8. Create a grievance object with date, department, priority and initial status.
9. Store the object in the collection.
10. Refresh the dashboard/table.
11. Allow the user to search the grievance by ID.
12. Allow authorized staff to update the complaint status.
13. Display the latest status and grievance details.
14. Continue until the user exits.
15. End.

Pseudocode

```
BEGIN
Initialize complaintList
Initialize citizenList
Initialize departmentList
Initialize complaintIdGenerator

Display main dashboard

WHEN Register Complaint is clicked:
   Read citizen details
   Read complaint details
   Validate mandatory fields
   IF validation fails:
      Display validation message
   ELSE:
      Generate complaint ID
      Create grievance object
      Assign department and priority
      Set status = "Submitted"
      Store grievance in complaintList
      Display success message
      Refresh complaint table

WHEN Search is clicked:
   Read complaint ID
   Search complaintList
   IF complaint exists:
      Display complaint details
```

```
    ELSE:
        Display "Complaint not found"

WHEN Update Status is clicked:
    Read complaint ID and selected status
    Locate grievance
    IF found:
        Update status
        Refresh table
    ELSE:
        Display error message

WHEN Exit is clicked:
    Close application
END
```

Department Assignment

```
IF category = ROADS
    department = Roads Department
ELSE IF category = WATER_SUPPLY
    department = Water Supply Department
```

Officer Assignment

```
Find officers belonging to selected department
IF officer available
    Assign officer
ELSE
    Display "Officer unavailable"
```

Multithreading

```
Create grievance processing threads
Start threads
Synchronize shared complaint data
Wait for processing
Notify waiting threads
```

Flowchart

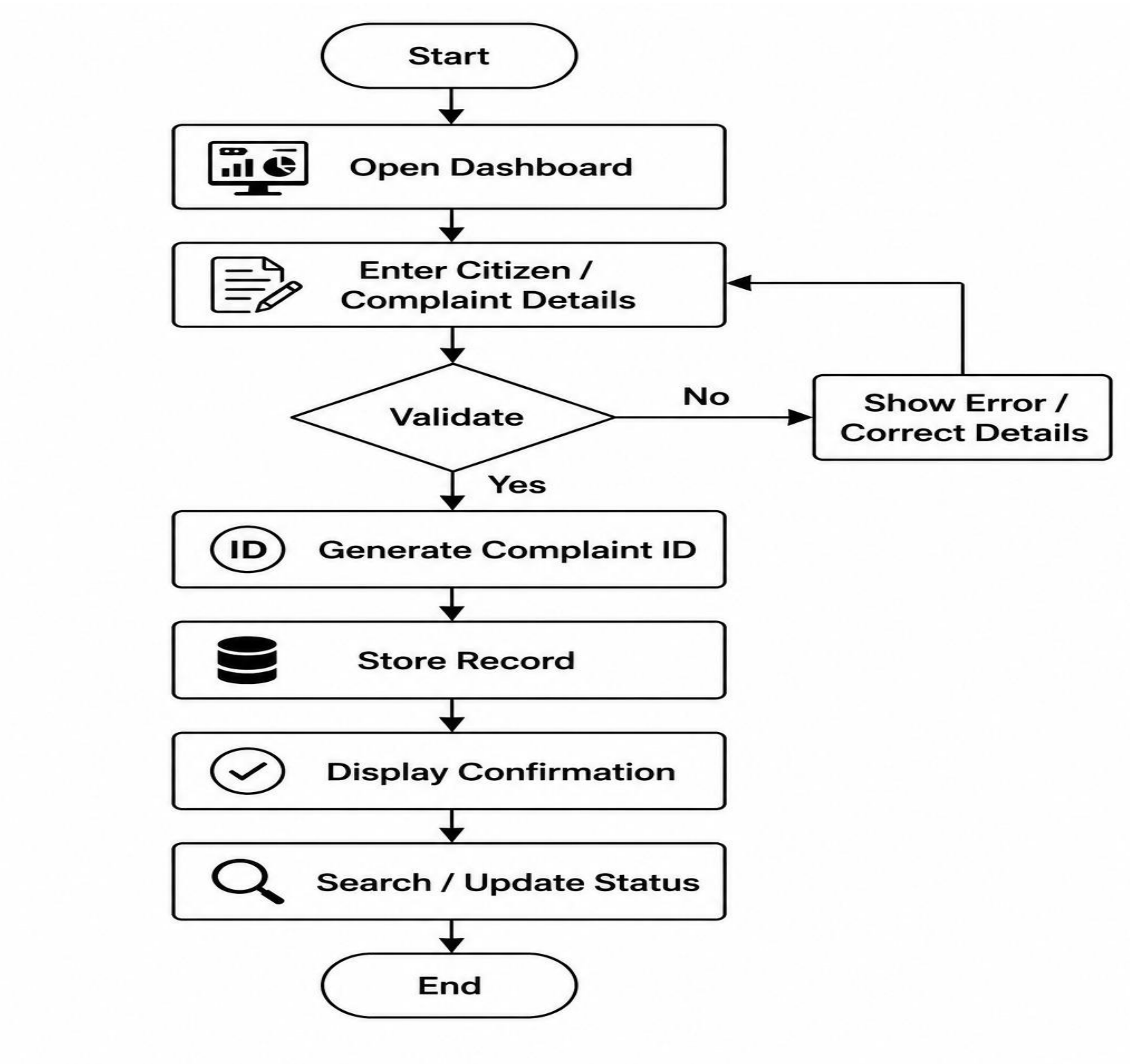


FIG 02 : Grievance Registration and Tracking Flowchart

Flow: Start → Open Dashboard → Enter Citizen/Complaint Details → Validate → Generate Complaint ID → Store Record → Display Confirmation → Search/Update Status → End.

Data Flow

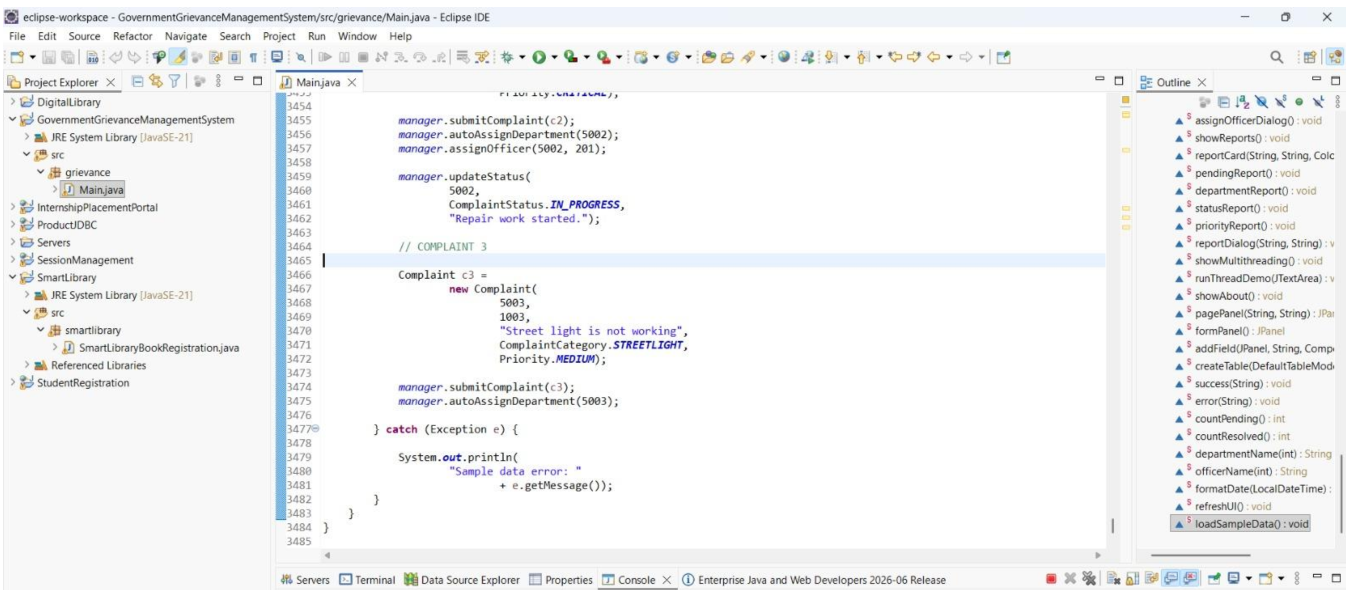
The user interface acts as the input layer. Event handlers transfer the validated values to the application logic, which creates or updates Java objects. The objects are maintained in collections and then rendered back into tables, labels or dialogs for the user.

6. Implementation / Source Code

Implementation

The Government Grievance Management System is implemented as a Java 21 desktop application using Eclipse IDE. The current implementation is designed around a single main Java source file under the grievance package, making it straightforward to compile and run for academic demonstration.

The implementation uses Java Swing and AWT for the graphical interface. Java collections provide temporary storage for citizen and grievance records. The interface contains form controls for entering information, action buttons for operations, tables or panels for displaying records, and visual status/priority indicators.



Source Code

The source code contains the principal components required for the application: Java entity representation, grievance registration, validation, complaint ID generation, department selection, priority handling, search, status update, table refresh and graphical presentation.

```
public class Main {

    static void showDashboard() {

        contentPanel.removeAll();

        JPanel panel =

            new JPanel(
```

```
        new BorderLayout(
            15,
            15));
panel.setOpaque(false);
JLabel welcome =
    new JLabel(
        "Welcome to the Grievance Portal");
welcome.setForeground(Color.WHITE);
welcome.setFont(
    new Font(
        "Segoe UI",
        Font.BOLD,
        27));
JPanel top =
    new JPanel(
        new FlowLayout(
            FlowLayout.LEFT));
top.setOpaque(false);
top.add(welcome);
panel.add(
    top,
    BorderLayout.NORTH);
contentPanel.add(
    panel,
```

```
            BorderLayout.CENTER);

        refreshUI();

    }
```

Environment Used

Environment Component	Configuration
Operating System	Windows
Programming Language	Java 21
IDE	Eclipse IDE
GUI Framework	Java Swing / AWT
Storage	Java collections / in-memory
Database	Not used in the current version
Project Structure	Java project → src → grievance → Main.java

Tools and Technologies Used

Tool / Technology	Purpose
Java 21	Core programming language and runtime
Eclipse IDE	Source-code editing, compilation and execution
Swing	Graphical user-interface components
AWT	Layout, color, font and event-support classes
Java Collections	In-memory complaint and citizen data management
Object-Oriented Programming	Entity modelling and modular logic

Implementation Modules

Main Interface Module – displays the dashboard and navigation controls.

Citizen Management Module – captures and validates citizen information.

Complaint Registration Module – creates a grievance record and generates an ID.

Department Assignment Module – associates the grievance with the responsible department.

Priority Management Module – records Normal, High or Critical priority.

Status Tracking Module – displays and updates complaint status.

Search Module – locates a grievance using its complaint identifier.

Data Management Module – stores and retrieves records using collections.

Testing Module – verifies the expected behaviour of each operation.

7. Test Cases and Expected/Actual Results

Test Case	Input / Action	Expected Result	Actual Result
TC01	Register valid citizen	Citizen registered successfully	Passed
TC02	Register duplicate citizen ID	Error message displayed	Passed
TC03	Submit complaint for valid citizen	Complaint submitted	Passed
TC04	Submit complaint for invalid citizen	Error displayed	Passed
TC05	Submit duplicate complaint	Duplicate complaint rejected	Passed
TC06	Submit complaint with empty description	Validation error displayed	Passed
TC07	Automatically assign department	Correct department assigned	Passed
TC08	Assign valid officer	Officer assigned successfully	Passed
TC09	Update complaint status	Status updated and history recorded	Passed
TC10	Track complaint using valid ID	Complaint details and history displayed	Passed
TC11	Track invalid complaint ID	Complaint not found message	Passed
TC12	Generate pending report	Pending complaints displayed	Passed

TC13	Generate department report	Department-wise statistics displayed	Passed
TC14	Generate status report	Status distribution displayed	Passed
TC15	Generate priority report	Priority distribution displayed	Passed
TC16	Run multithreading demonstration	Multiple threads execute successfully	Passed

8. EXECUTION SCREENSHOTS / OUTPUTS

This section follows the evidence-oriented structure of the reference format. Replace each placeholder with an actual screenshot from the completed Java application and retain the corresponding figure number and caption.

Main Dashboard

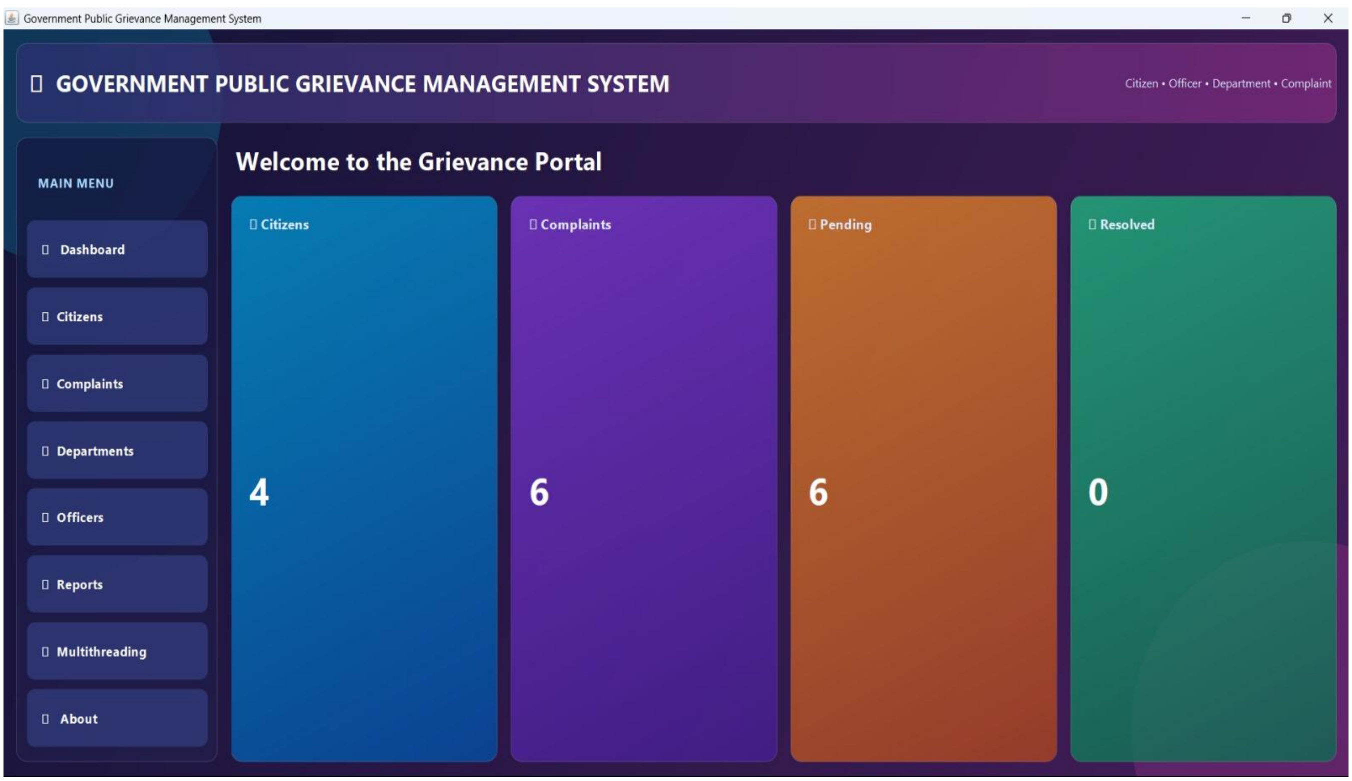


FIG 03 : Government Grievance Management System – Main Dashboard

The dashboard should show the major navigation or operational controls available to the user.

Citizen and Grievance Registration

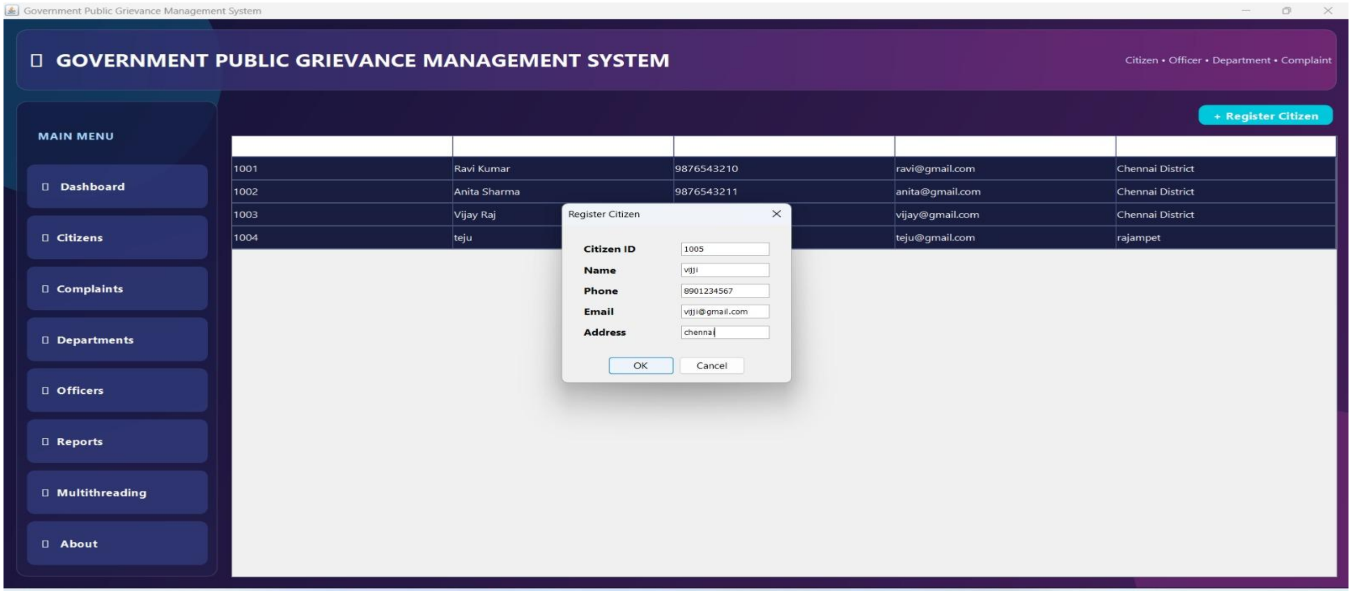


FIG 04 : Citizen and Grievance Registration Form

The screenshot should show citizen details, complaint category, description, location, department and priority fields.

Complaint Registration Confirmation

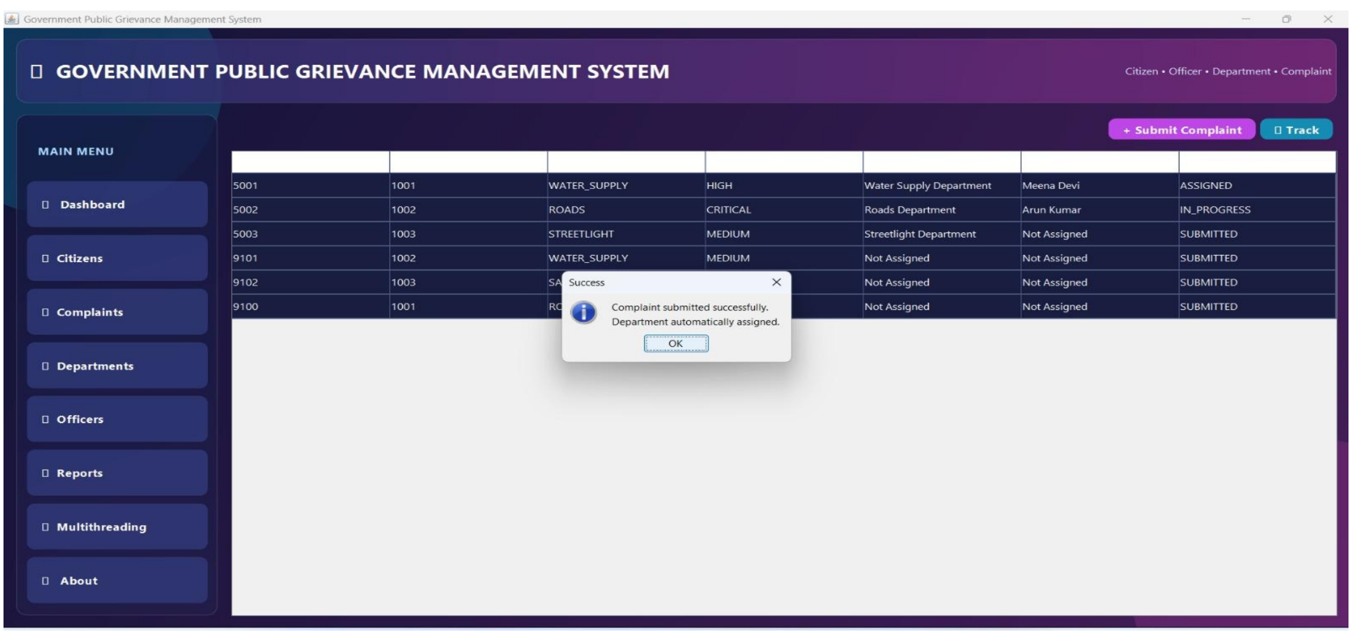


FIG 05 : Successful Complaint Registration

Show the confirmation message and generated complaint/reference ID.

Complaint List / Management View

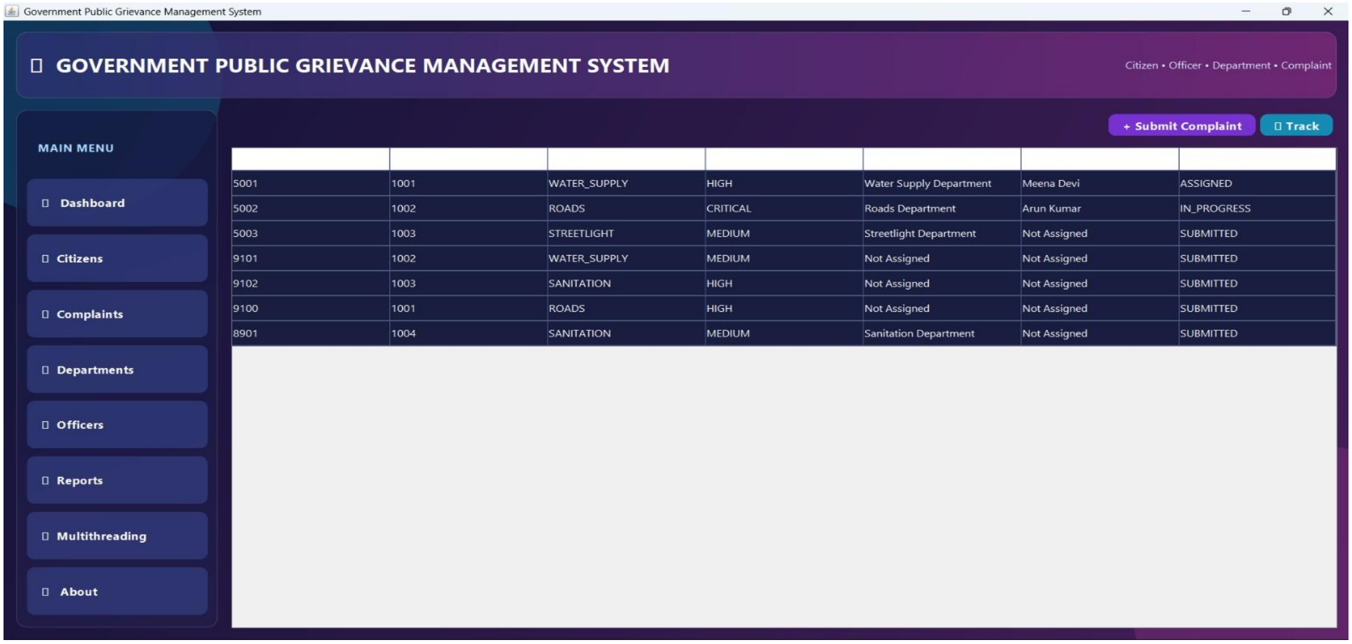


FIG 06 : Registered Grievances Management View

Show multiple complaint records with ID, category, department, priority and status.

Complaint Search

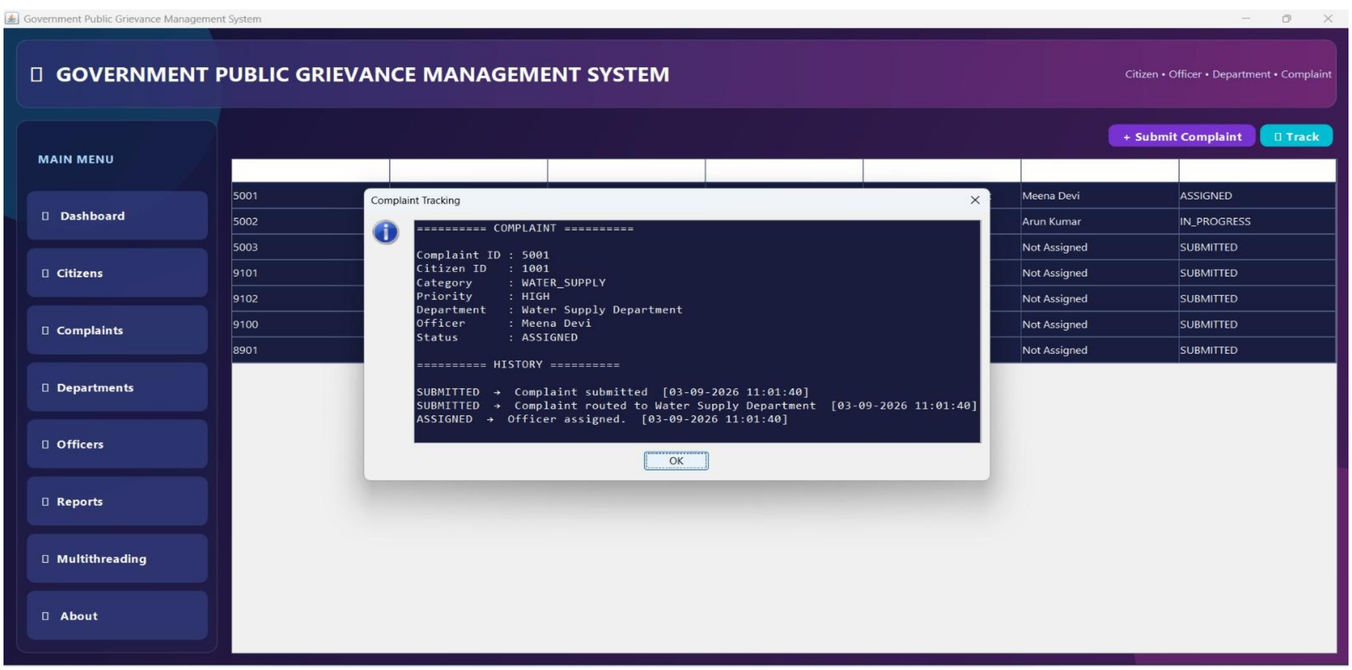


FIG 07 : Complaint Search and Tracking

Show a search using a valid complaint ID and the corresponding grievance details.

Status Update

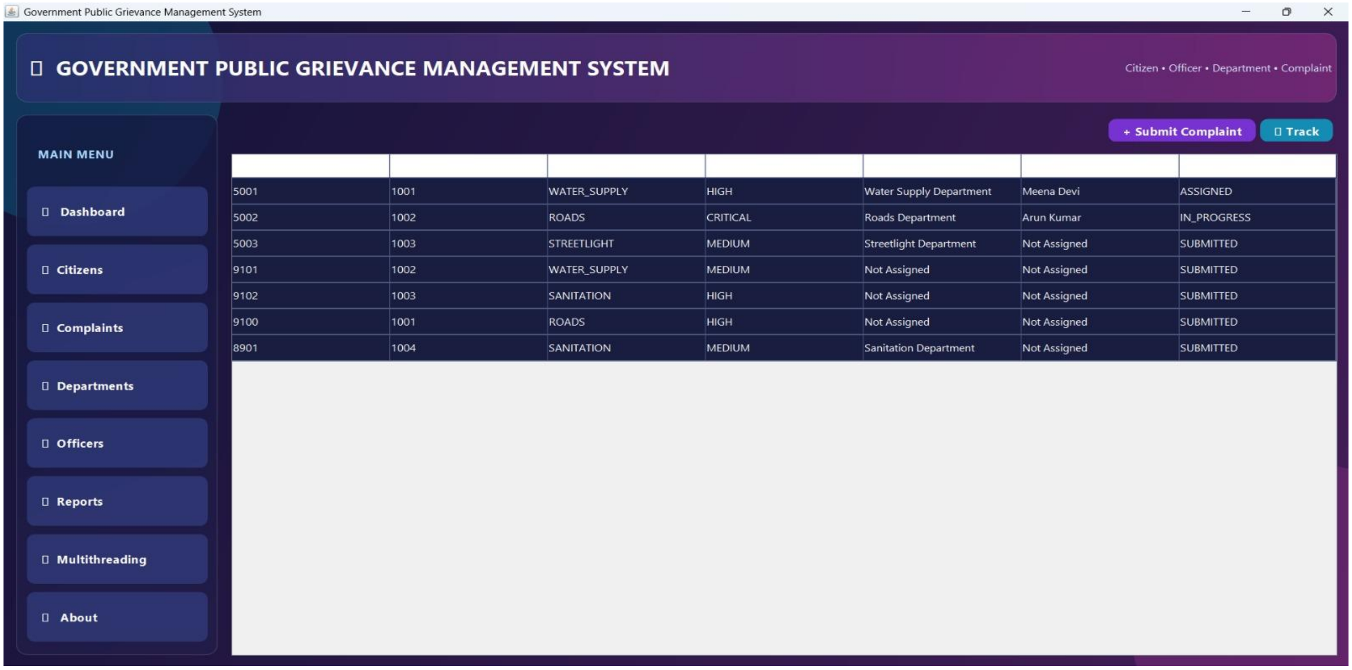


FIG 08: Complaint Status Update

Show a grievance changing from Submitted to In Progress, Resolved or Closed.

Department Management

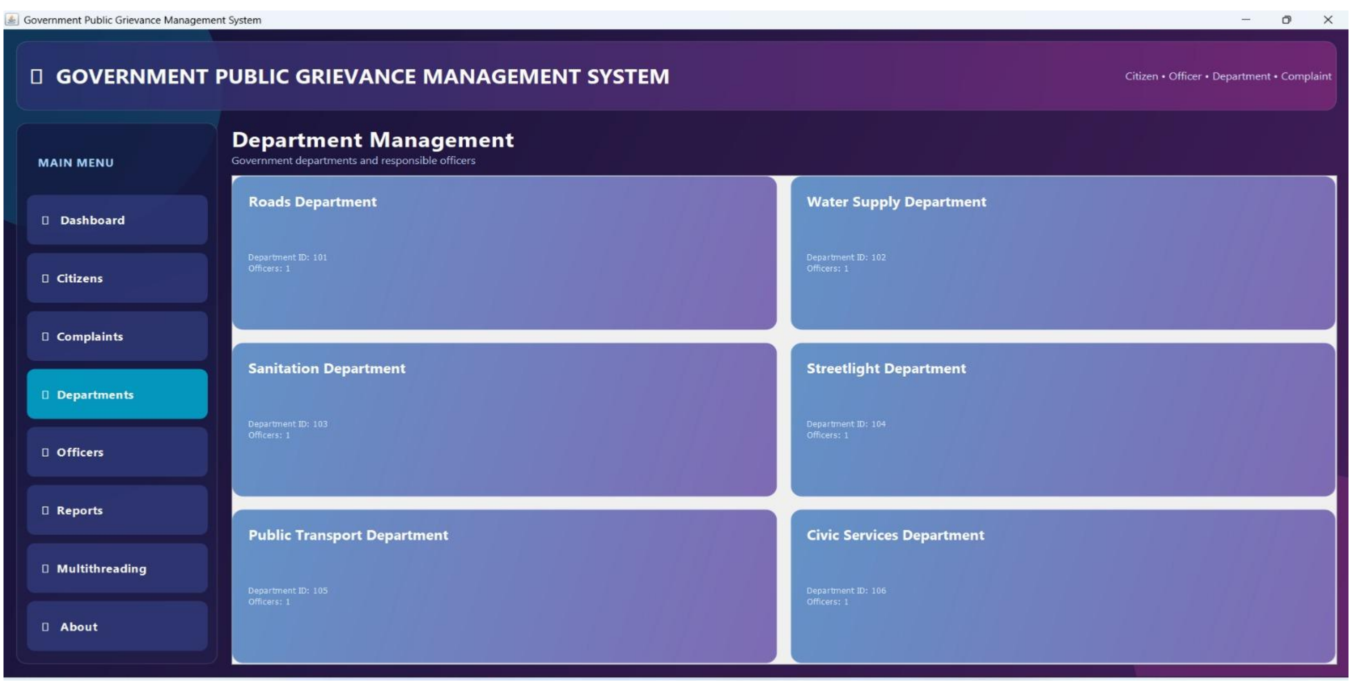


FIG 09 : Department Management

Show how high-priority or critical grievances are visually distinguished.

Officer Management

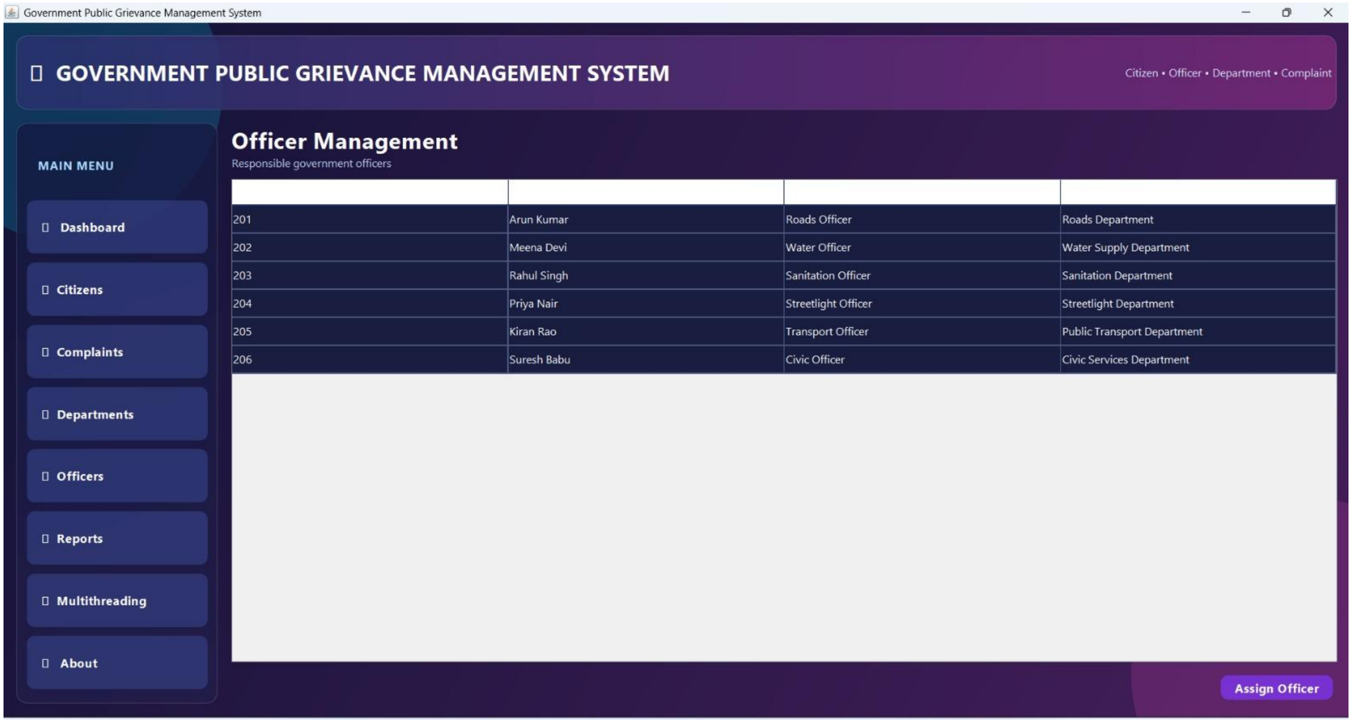


FIG 10 : Officer Management

Reports and Analysis

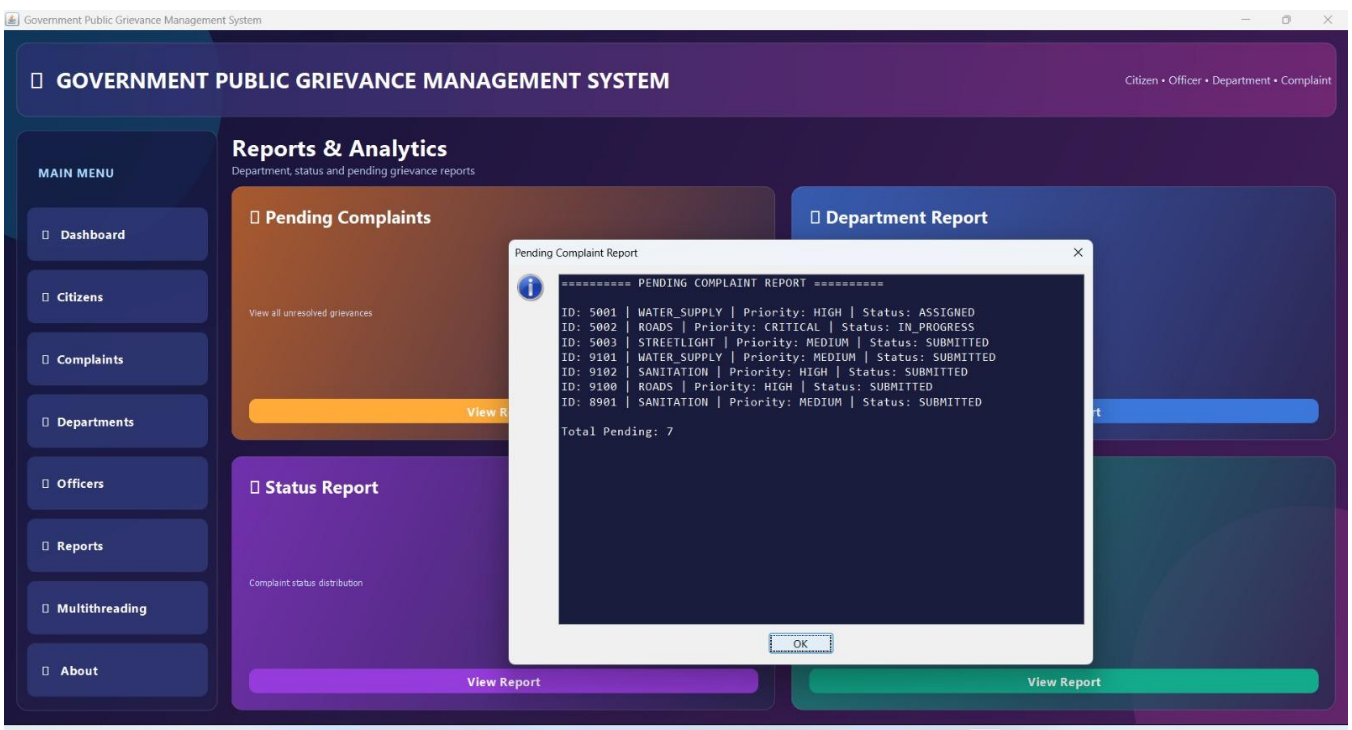


FIG 11 : Reports and analysis

Multithreading and Synchronization

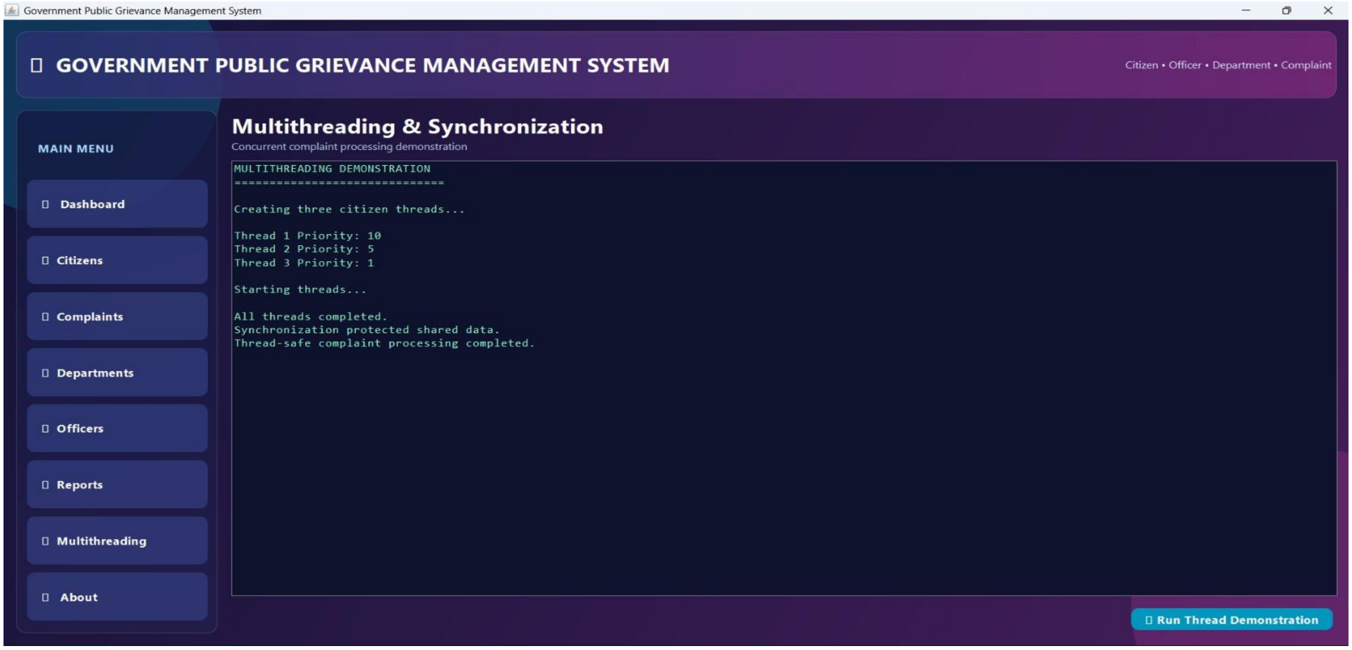


FIG 12 : Multithreading and Synchronization

9. Analysis and Discussion

The implemented system successfully provides a structured approach for managing government grievances. The use of ArrayList allows citizens and complaints to be maintained dynamically. HashSet is used for duplicate complaint detection, while HashMap provides efficient access to departments and officers using their IDs. Object-oriented programming improves the organization of the system. The User class acts as a parent class, while Citizen, Officer, Supervisor, and Administrator demonstrate inheritance and polymorphism.

Custom exceptions are used to handle invalid inputs, duplicate complaints, unavailable departments, and invalid complaint statuses. The system also demonstrates multithreading. Multiple complaint-processing threads can operate concurrently, while synchronization ensures that shared data is accessed safely.

The graphical interface makes the system easier to use by providing separate screens for citizens, complaints, departments, officers, reports, and multithreading. One limitation of the current implementation is that the data is maintained in memory using Java collections. Therefore, the data is not permanently stored after the application is closed.From my contribution perspective, the Department, Officer, Reports and Multithreading modules helped me understand how different system components interact during grievance processing.

Department mapping ensures that complaints reach the appropriate department, while officer assignment supports responsibility allocation. Report generation provides useful summaries, and multithreading with synchronization demonstrates safe concurrent processing.

10. Conclusion

The Government Grievance Management System successfully demonstrates the development of a Java-based application for managing public complaints. The system provides citizen registration, complaint submission, automatic department assignment, officer assignment, complaint tracking, status management, duplicate detection, reporting, and multithreaded processing.

The project also demonstrates important Java programming concepts such as object-oriented programming, collections, generics, exception handling, synchronization, multithreading, and GUI development using Swing. Overall, the system provides a simple and organized approach to public grievance management and demonstrates how Java programming concepts can be applied to a practical real-world application.My contribution to department management, officer assignment, reporting, multithreading and synchronization helped demonstrate how Java can be applied to coordinate and monitor grievance-processing activities.

11. Individual Contribution of Group Members

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
N. Vijetha 192311258	Citizen and Complaint Management	Developed User, Citizen, registration, Complaint, ComplaintHistory, and complaint submission modules	Tested citizen registration, validation, complaint submission, duplicate detection, and complaint tracking	Prepared citizen and complaint module documentation	50%
N.Tejaswitha 192371057	Department, Officer, Reports and Multithreading	Developed Department, Officer, assignment, reports, multithreading, and synchronization modules	Tested department routing, officer assignment, reports, threads, and synchronization	Prepared department/officer documentation, testing, analysis, and conclusion	50%

Total Group Contribution: 100%

12. References

- Oracle Corporation, Java Platform, Standard Edition (Java SE) 21 Documentation, 2023–2026.
- Oracle Corporation, Java SE 21 API Specification, 2023–2026.
- Oracle Corporation, Java Development Kit 21 Release Notes, September 2023.
- Oracle Corporation, The Collections Framework – Java SE 21, 2023–2026.
- K. Scarfone, M. Souppaya and D. Dodson, Secure Software Development Framework (SSDF) Version 1.1, NIST Special Publication 800-218, National Institute of Standards and Technology, 2022.
- OWASP Foundation, Application Security Verification Standard (ASVS), Version 5.0, 2025.
- National Institute of Standards and Technology (NIST), Secure Software Development Framework (SSDF) Project Publications, updated 2026.
- Oracle Corporation, Java Core Libraries Developer Documentation – JDK 21, 2023–2026.

13. ONE-PAGE INDIVIDUAL REFLECTION

Individual Reflection – N.Tejaswitha

Working on the **Government Grievance Management System** gave me practical experience in developing a Java-based application for a real-world public-service problem. My main contribution to the project was focused on the **Department Management, Officer Management, Reports, Multithreading and Synchronization modules**. I also contributed to testing, analysis and documentation of these modules.

Through the **Department Management module**, I learned how different complaint categories can be mapped to the appropriate government departments. This helped me understand the importance of proper department allocation in reducing delays in grievance processing. In the **Officer Management module**, I worked with officer details and officer assignment, which helped me understand how complaints can be assigned to responsible officers based on their departments.

I also gained experience in developing **Reports and Analytics**. The system provides pending-complaint, department-wise, status-wise and priority-wise reports. Working on these reports helped me understand how stored grievance data can be processed and presented in a useful form for monitoring and decision-making.

The **Multithreading and Synchronization module** was an important learning experience. I understood how multiple threads can execute concurrently and why synchronization is required when shared grievance data is accessed by different threads. I also learned how synchronization helps prevent conflicts and maintain safe processing of concurrent operations.

During testing, I verified **department routing, officer assignment, report generation, multithreading and synchronization**. Testing helped me understand the importance of checking both successful and incorrect operations and improved my problem-solving skills when identifying and resolving errors.

The project is closely related to **SDG 16 – Peace, Justice and Strong Institutions**, as an organized grievance management system can support transparent and accountable public-service processes. It also relates to **SDG 9 – Industry, Innovation and Infrastructure** through the use of technology to improve service management and **SDG 11 – Sustainable Cities and Communities** through better handling of civic complaints. Overall, this project improved my understanding of **Java object-oriented programming, collections, exception handling, multithreading, synchronization and GUI-based application development**. It also improved my teamwork, testing and documentation skills. I learned how individual modules can be integrated into a complete system. In the future, the application can be improved by adding **MySQL/JDBC-based permanent storage, authentication, notifications and a web-based interface**.

PLAGIARISM CHECKER :